



Chapter11 DSP Algorithm Implementation

- *Structure Simulation and Verification by MATLAB*
- *Computation of DFT*
 - ✓ *Fast Fourier Transformation —— FFT*
 - ✓ *DFT and IDFT Computation Using MATLAB*
- *Number Representation*





11.2 Structure simulation Using MATLAB

- **Program:**

**direct2.m, 11_1.m, 11_2.m, 11_3.m,
11_4.m, 11_5.m, 11_6.m, 11_7.m**

11.3 Computation of the DFT

11.3.1 Goertzel's Algorithm

11.3.2 Cooley-Tukey FFT Algorithms



- **FFT**--Fast Fourier Transformation, not a new transformation, but a fast algorithm to calculate the DFT.
- J.W.Cooley and J.W.Tukey are given credit for bringing the FFT to the world in their paper :“An algorithm for the machine calculation of complex Fourier Series”, Mathematics Computation, Vol.19, 1965.
- IEEE electroacoustic journal published an album on fast Fourier transform algorithms in 1967, starting the history of fast transform.
- Magazine in January 1992 and IEEE ICASSP 92 have featured articles or reports on commemorating the release of FFT albums.
- In April 1994, the album "Understanding Fourier Technique" is published with the purpose of popularizing and promoting FFT technology

Development of FFT



- The traditional fast Fourier transform algorithm is recursive decomposition algorithm, including basis 2, basis 4, basis 8 and split basis FFT.
- The radix K-FFT (also known as convolutional FFT algorithm C-FFT) proposed by Stasinisik is a new development of other radix FFT algorithms.
- Frigo and Johnson developed the FFTW free library in 2005 to provide a solution for running the FFT algorithm on a variety of hardware platforms.

.....

- **DFT and IDFT definition:**

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, 0 \leq k \leq N-1$$

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-kn}, 0 \leq n \leq N-1$$

Where, $W_N = e^{-j2\pi/N}$

- **Direct computation of all N samples of $\{X[k]\}$ requires N^2 complex multiplications and $N(N-1)$ complex additions.**



The properties of twiddle factor W_N :

- **Symmetry:** $(W_N^{nk})^* = W_N^{-nk}$
- **Periodicity:** $W_N^{nk} = W_N^{(n+N)k} = W_N^{n(k+N)}$
- **Reduction:** $W_N^{nk} = W_{mN}^{nmk} \quad W_N^{nk} = W_{N/m}^{nk/m}$

$$W_N^{n(N-k)} = W_N^{(N-n)k} = W_N^{-nk}, W_N^{N/2} = -1, W_N^{(k+N/2)} = -W_N^k$$

Decimation-in-Time FFT



- Given a sequence $x[n]$ whose length is $N=2^k$, k is an integer.
- Divided the sequence into **even index sequences** and **odd index sequences**:

$$\begin{aligned} x[2r] &= x_0[r] \\ x[2r+1] &= x_1[r] \end{aligned}, r = 0, 1, 2, \dots, \frac{N}{2} - 1$$

- Since $x[n]$ is first decimated to form a set of subsequences before the DFT is computed, this computation schemes are called **decimation-in-time (DIT) FFT** algorithms.

Decimation-in-Time FFT



$$\begin{aligned} X[k] &= DFT\{x[n]\} = \sum_{n=0}^{N-1} x[n]W_N^{nk} = \sum_{r=0}^{N/2-1} x[2r]W_N^{2rk} + \sum_{r=0}^{N/2-1} x[2r+1]W_N^{(2r+1)k} \\ &= \sum_{r=0}^{N/2-1} x_0[r]W_N^{2rk} + W_N^k \sum_{r=0}^{N/2-1} x_1[r]W_N^{2rk} = \sum_{r=0}^{N/2-1} x_0[r]W_{N/2}^{rk} + W_N^k \sum_{r=0}^{N/2-1} x_1[r]W_{N/2}^{rk} \end{aligned}$$

- **We get:** $X[k] = X_0[k] + W_N^k X_1[k], k = 0, 1, 2, \dots, N/2 - 1$

Where $X_0[k]$ and $X_1[k]$ is **$N/2$** -point DFT, so we get only first half $N/2$ -point result of $X[k]$.

Decimation-in-Time FFT



- Based on the periodicity of W_N , we can get:

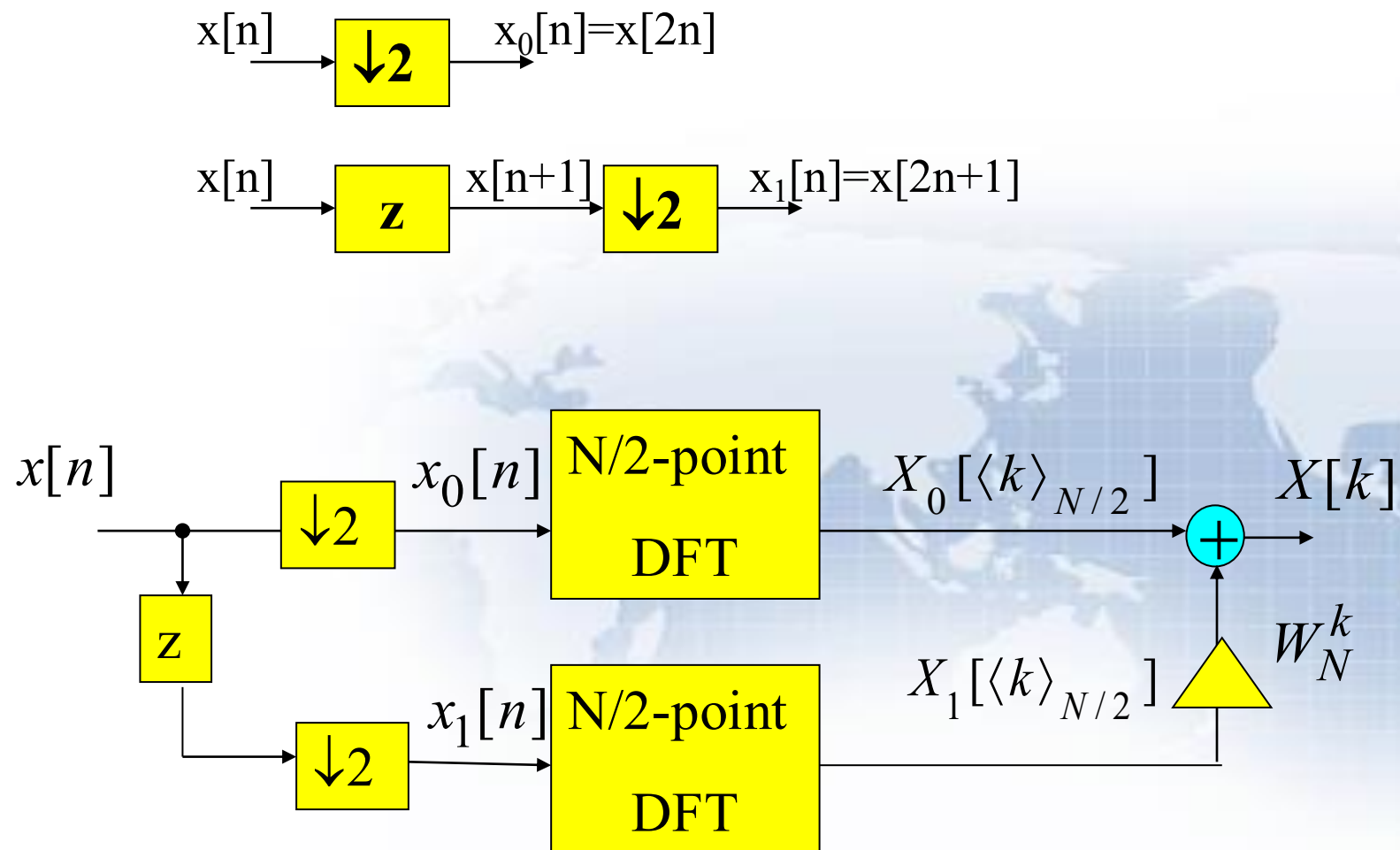
$$X_0\left[\frac{N}{2} + k\right] = \sum_{r=0}^{N/2-1} x_0[r] W_{N/2}^{r(\frac{N}{2}+k)} = \sum_{r=0}^{N/2-1} x_0[r] W_{N/2}^{rk} = X_0[k]$$

$$X_1\left[\frac{N}{2} + k\right] = X_1[k] \quad \text{And:} \quad W_N^{(\frac{N}{2}+k)} = W_N^{\frac{N}{2}} W_N^k = -W_N^k$$

So:

$$X\left[\frac{N}{2} + k\right] = X_0\left[\frac{N}{2} + k\right] + W_N^{\frac{N}{2}+k} X_1\left[\frac{N}{2} + k\right] = X_0[k] - W_N^k X_1[k], \quad k = 0, 1, 2, \dots, N/2 - 1$$

- Block-diagram interpretation:



Decimation-in-Time FFT



- The computation of N-point DFT by two methods:

DFT computation	Complex Addition	Complex Multiplication
Direct computation	$N^2 - N \approx N^2$	N^2
DIT to two N/2-point DFT	$(N^2/2) + N$	$(N^2/2) + N$

Decimation-in-Time FFT



- Continuing the process we can express $X_0[k]$ and $X_1[k]$ as a weighted combination of two $(N/4)$ -point DFTs.

$$X_0[k] = X_{00}[\langle k \rangle_{N/4}] + W_{N/2}^k X_{01}[\langle k \rangle_{N/4}], \\ 0 \leq k \leq (N/2) - 1$$

where $X_{00}[k]$ and $X_{01}[k]$ are the $(N/4)$ -point DFTs of the $(N/4)$ -length sequences:

$$\mathbf{x}_{00}[\mathbf{n}] = \mathbf{x}_0[2\mathbf{n}] \text{ and } \mathbf{x}_{01}[\mathbf{n}] = \mathbf{x}_0[2\mathbf{n}+1]$$

Decimation-in-Time FFT



- Likewise, we can express

$$X_1[k] = X_{10}[\langle k \rangle_{N/4}] + W_{N/2}^k X_{11}[\langle k \rangle_{N/4}],$$
$$0 \leq k \leq (N/2) - 1$$

where $X_{10}[k]$ and $X_{11}[k]$ are the $(N/4)$ -point DFTs of the $(N/4)$ -length sequences:

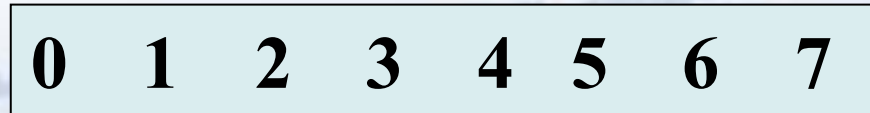
$$\mathbf{x}_{10}[\mathbf{n}] = \mathbf{x}_1[2\mathbf{n}] \text{ and } \mathbf{x}_{11}[\mathbf{n}] = \mathbf{x}_1[2\mathbf{n}+1]$$

Decimation-in-Time FFT



- The following figure shows the time domain decomposition used in the FFT:

1 signal of 8 points



2 signal of 4 points



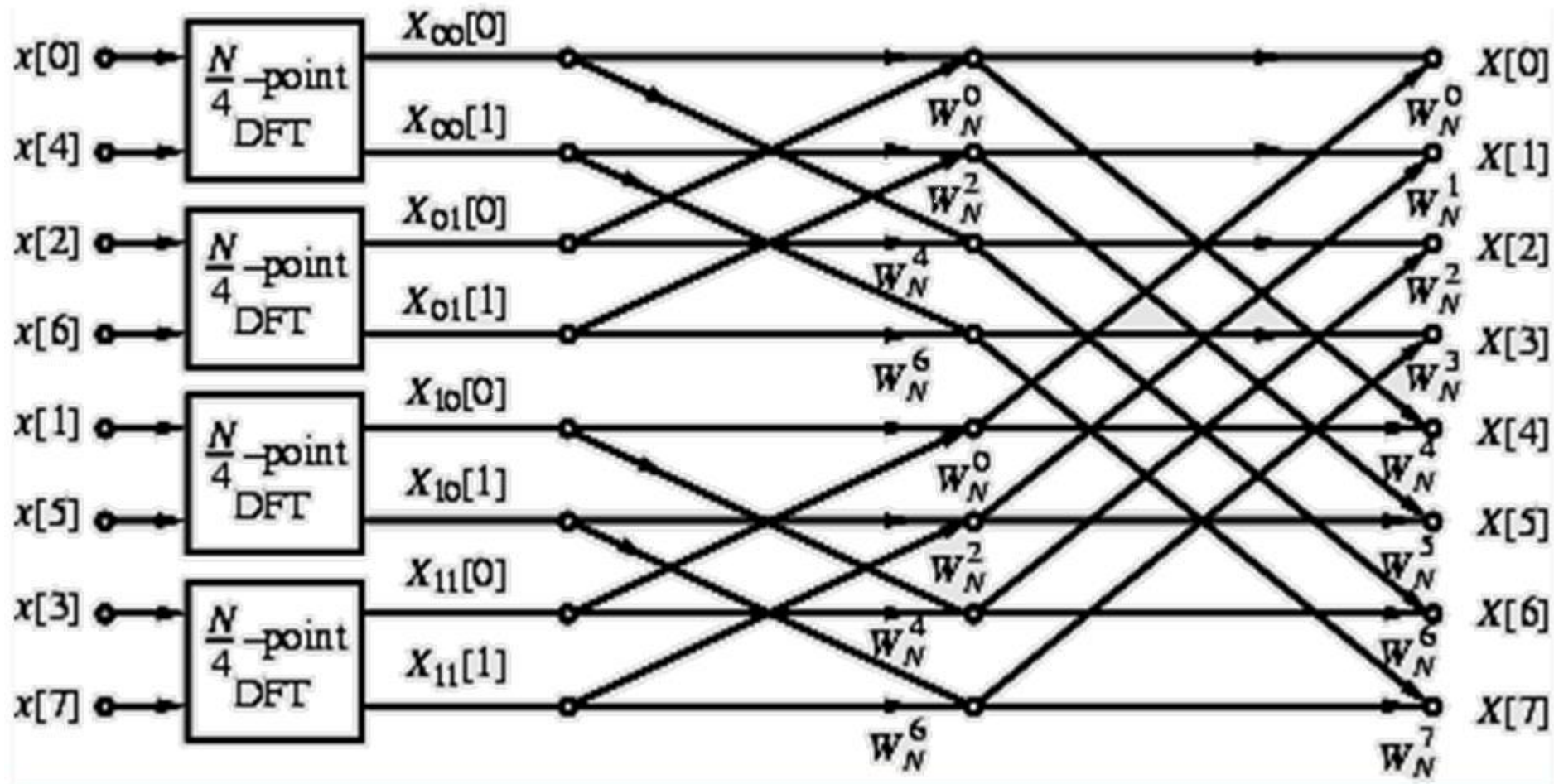
4 signals of 2 points



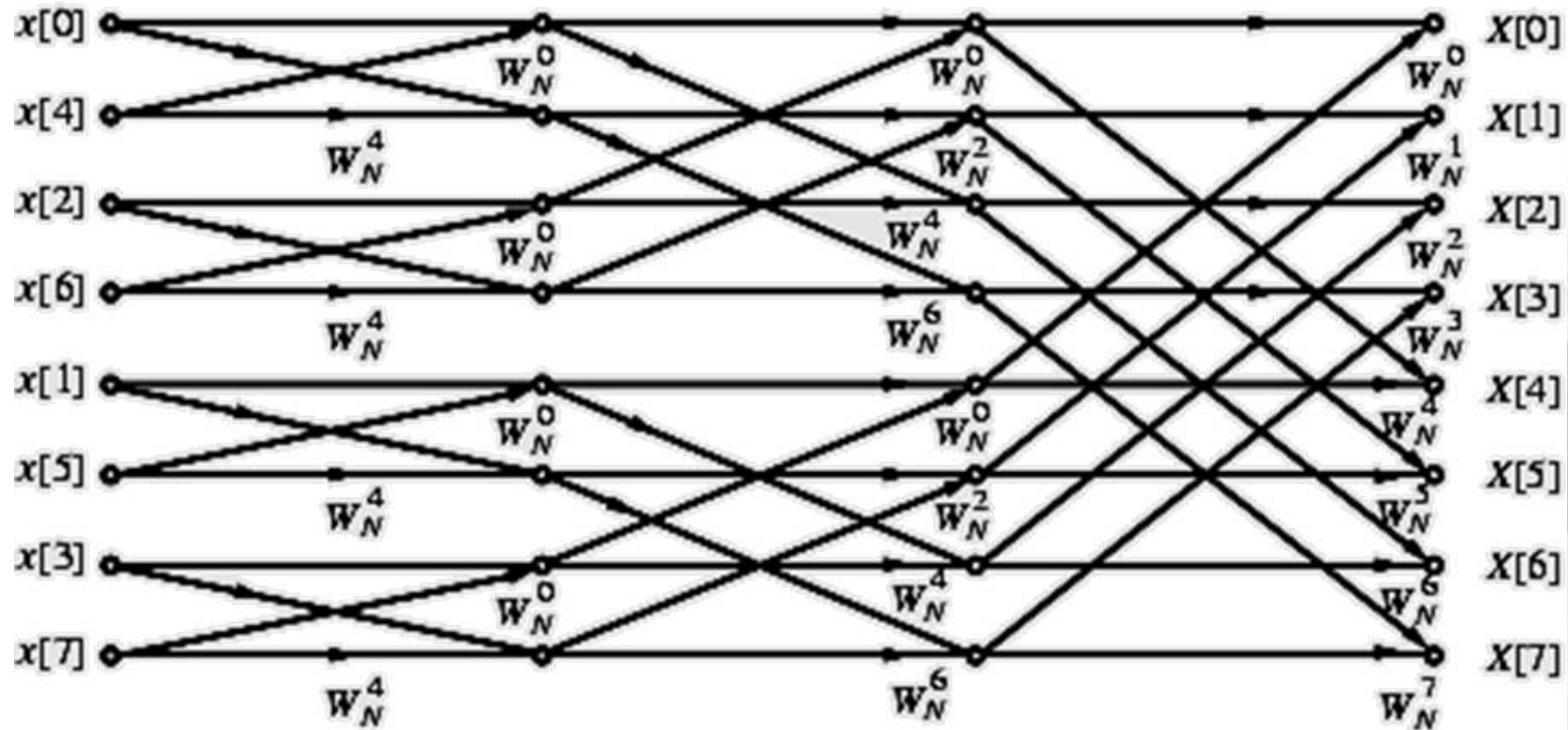
Decimation-in-Time FFT



- Flow-graph representation



- Complete flow-graph of the 8-point DFT is shown below



Each stage of the DFT computation process employs the same basic computational module which is called a **butterfly computation**.

Decimation-in-Time FFT



- Total number of complex multiplications and additions to compute all 8 DFT samples is equal to $8 + 8 + 8 = 24 = 8 \times 3$.
- In the general case when $N=2^k$, number of stages for the computation of the (2^k) -point DFT in the fast algorithm will be $k=\log_2 N$.
- Total number of complex multiplications to compute all N DFT samples is $N(\log_2 N)/2$
- Total number of complex additions to compute all N DFT samples is $N(\log_2 N)$

Decimation-in-Time FFT



Number of Points	Direct Computation of the DFT		Radix-2 FFT	
	Complex Multiplies	Complex Additions	Complex Multiplies	Complex Additions
N	N^2	$N^2 - N$	$(N/2)\log_2 N$	$N\log_2 N$
4	16	12	4	8
16	256	240	32	64
64	4096	4032	192	384
256	65536	65280	1024	2048
1024	1048576	1047552	5120	10240

Decimation-in-Time FFT



- In developing the count, multiplications with $W_N^0 = 1$ and $W_N^{N/2} = -1$ have been assumed to be complex
- We have not taken advantage of the symmetry property:

$$W_N^{(N/2)+k} = -W_N^k$$

These properties can be exploited to reduce the computational complexity further.

Decimation-in-Time FFT



- The FFT algorithm described here also is efficient with regard to memory requirements;
- At the end of computation at any stage, output variables can be stored in the **same registers** previously occupied by the corresponding input variables;
- This type of memory location sharing is called **in-place computation** resulting in significant savings in overall memory requirements.

Decimation-in-Time FFT



- In the DFT computation scheme outlined, the DFT samples $X[k]$ appear at the output in a sequential order while the input samples $x[n]$ appear in a different order;
- Thus, a sequentially ordered input $x[n]$ must be reordered appropriately before the fast algorithm described by this structure can be implemented.

- **bit-reversed order**

if $n = (b_2b_1b_0)$ represents the index of $x[n]$, then $m = (b_0b_1b_2)$ is the correct index for FFT input samples. i.e.

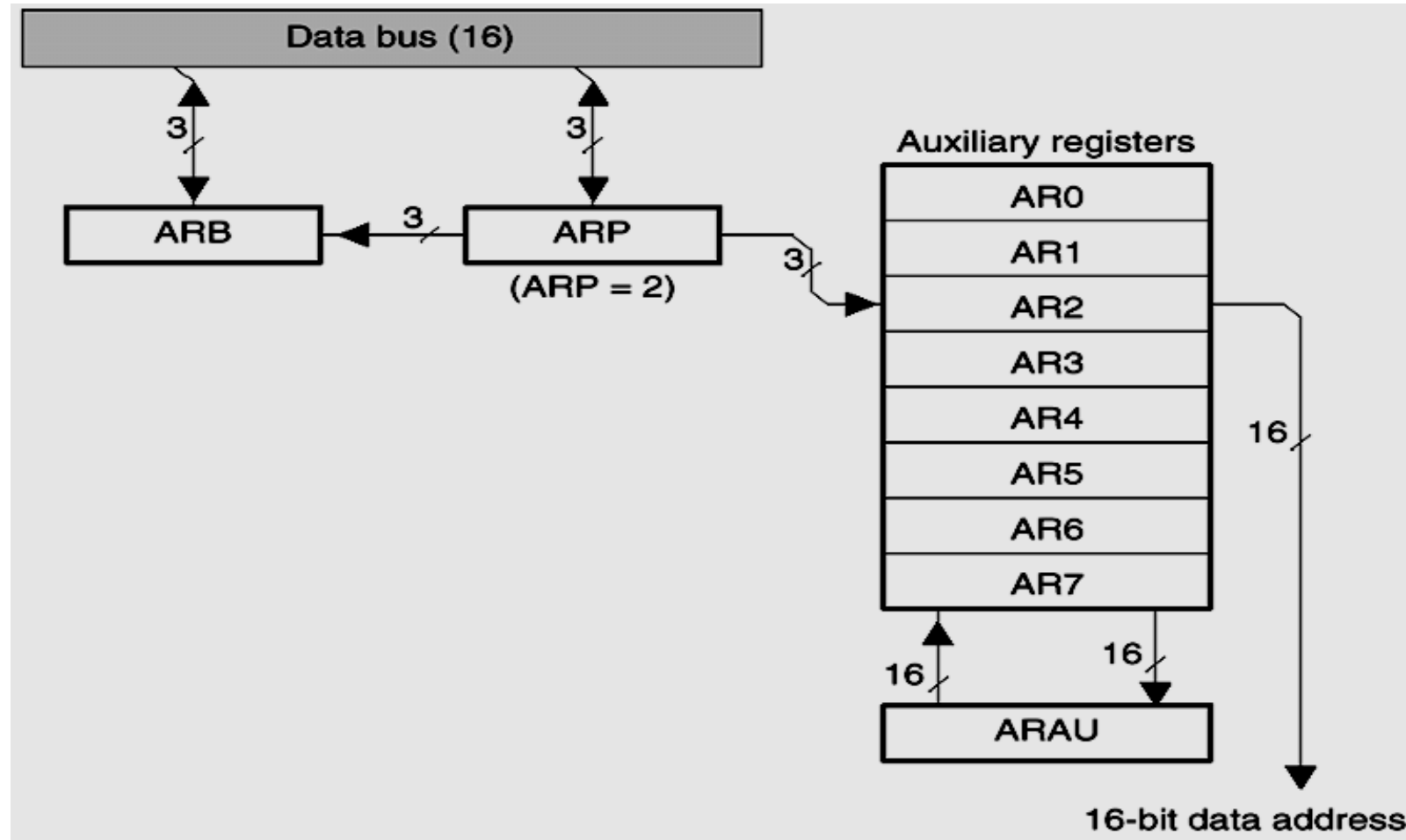
n: 000 001 010 011 100 101 110 111

m: 000 100 010 110 001 101 011 111



original order	original address	bit-reversed address	bit-reversed order
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7

Indirect Addressing in DSP



- Bit-reversed addressing enhances execution speed and program memory for FFT.

Decimation-in-frequency FFT



- Given a sequence $x[n]$ whose length is $N=2^L$, L is an integer. And dividing the sequence into two half sequences and compute its DFT:

$$\begin{aligned} X[k] &= DFT\{x[n]\} = \sum_{n=0}^{N-1} x[n]W_N^{nk} = \sum_{n=0}^{N/2-1} x[n]W_N^{nk} + \sum_{n=N/2}^{N-1} x[n]W_N^{nk} \\ &= \sum_{n=0}^{N/2-1} x[n]W_N^{nk} + \sum_{n=0}^{N/2-1} x[n + \frac{N}{2}]W_N^{(n+\frac{N}{2})k} \\ &= \sum_{n=0}^{N/2-1} [x[n] + x[n + \frac{N}{2}]]W_N^{\frac{N}{2}k} gW_N^{nk}, k = 0, 1, \dots, N-1 \end{aligned}$$

Decimation-in-frequency FFT



Because $W_N^{\frac{N}{2}} = -1$, and $W_N^{\frac{N}{2}k} = (-1)^k$, then:

$$X[k] = \sum_{n=0}^{N/2-1} [x[n] + (-1)^k x[n + \frac{N}{2}]] g W_N^{nk}, k = 0, 1, \dots, N-1$$

When k is even number:

$$X[2r] = \sum_{n=0}^{N/2-1} [x[n] + x[n + \frac{N}{2}]] g W_N^{2rn}, k = 0, 1, \dots, N-1$$

When k is odd number:

$$X[2r+1] = \sum_{n=0}^{N/2-1} [x[n] - x[n + \frac{N}{2}]] g W_N^{(2r+1)n}, k = 0, 1, \dots, N-1$$

Decimation-in-frequency FFT



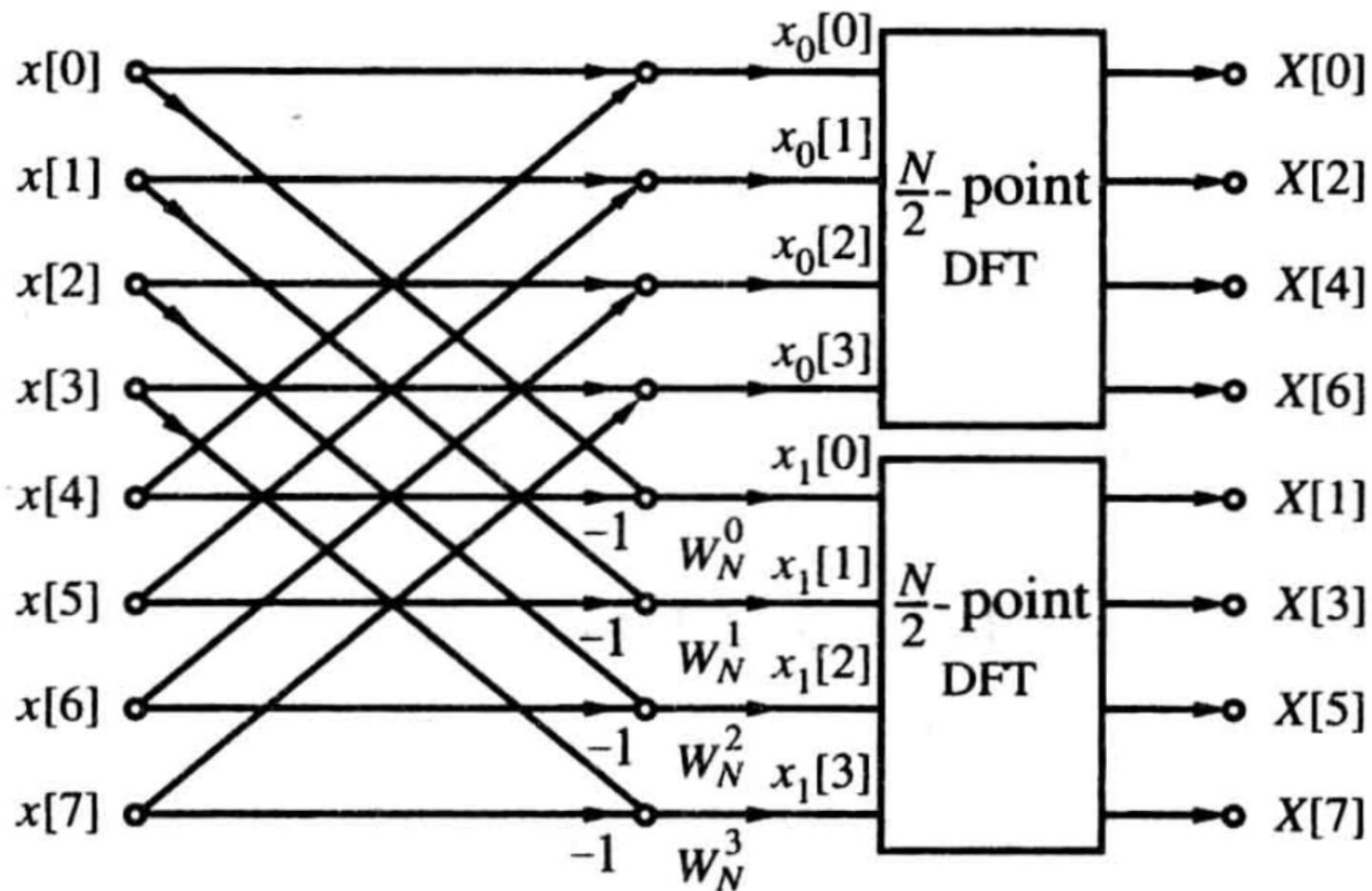
Based on $W_N^{2rk} = W_{N/2}^{rk}$, above formula equals to:

$$X[2r] = \sum_{n=0}^{N/2-1} [x[n] + x[n + \frac{N}{2}]] g W_{N/2}^{rn}, r = 0, 1, \dots, N/2 - 1$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} \{ [x[n] - x[n + \frac{N}{2}]] g W_N^n \} W_{N/2}^{nr}, r = 0, 1, \dots, N/2 - 1$$

◆ **Process is continued until the smallest DFTs are 2-point DFTs.**

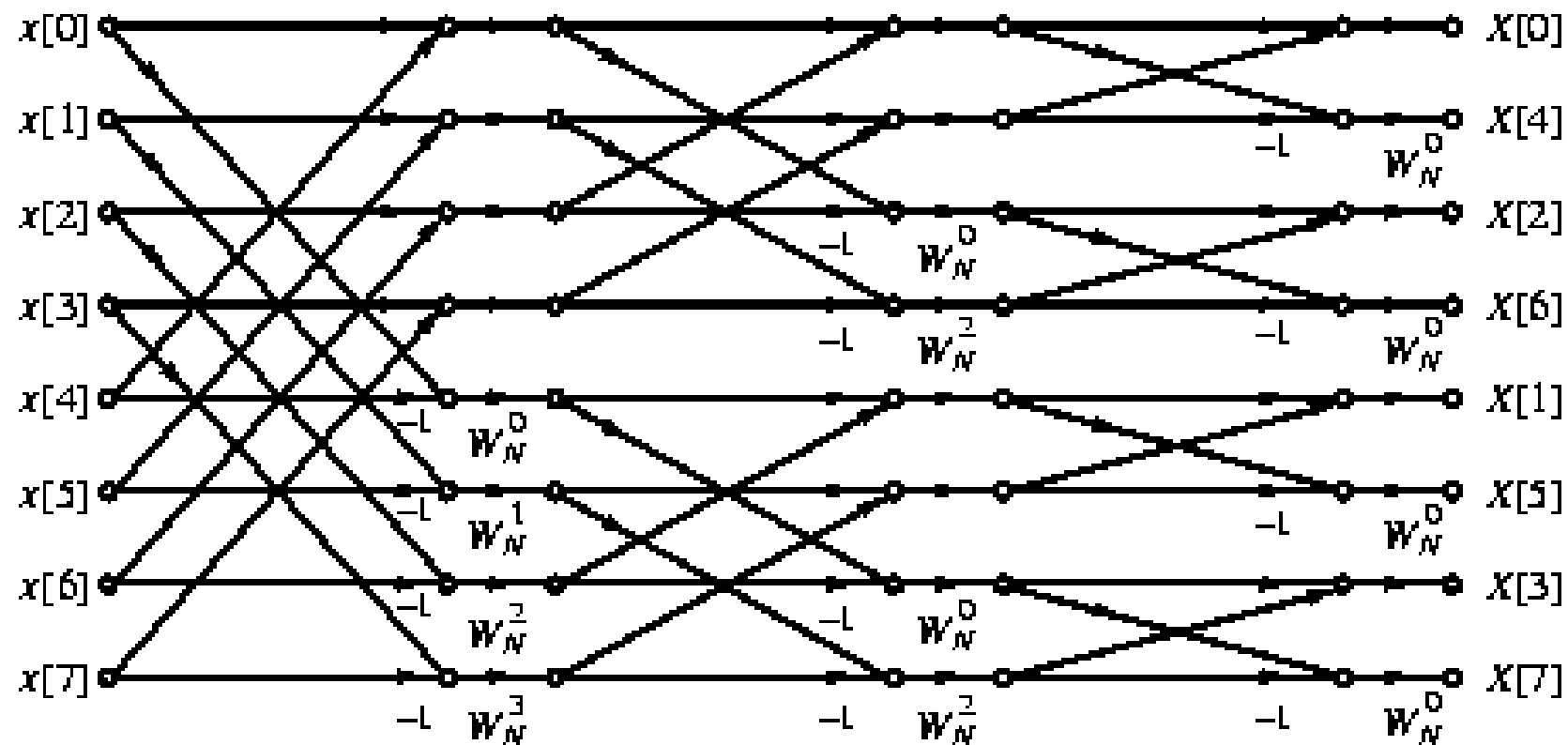
Decimation-in-frequency FFT



Decimation-in-frequency FFT



- Complete flow-graph of the **DIF** FFT computation scheme for $N = 8$.



Decimation-in-frequency FFT



- **Computational complexity of the radix-2 DIF FFT algorithm is same as that of the DIT FFT algorithm.**
- **Various forms of DIF FFT algorithm can similarly be developed.**
- **It can be easily seen that the flow-graph of DIF FFT is the transpose of the flow-graph of DIT FFT, and vice-versa.**

Inverse DFT Computation



- An FFT algorithm for computing the DFT samples can also be used to calculate the inverse DFT (IDFT) efficiently.
- Consider a length- N sequence $x[n]$ with an N -point DFT $X[k]$:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-nk}$$

Inverse DFT Computation



- **Multiplying both sides by N and taking the complex conjugate we get:**

$$Nx^*[n] = \sum_{k=0}^{N-1} X^*[k] W_N^{nk}$$

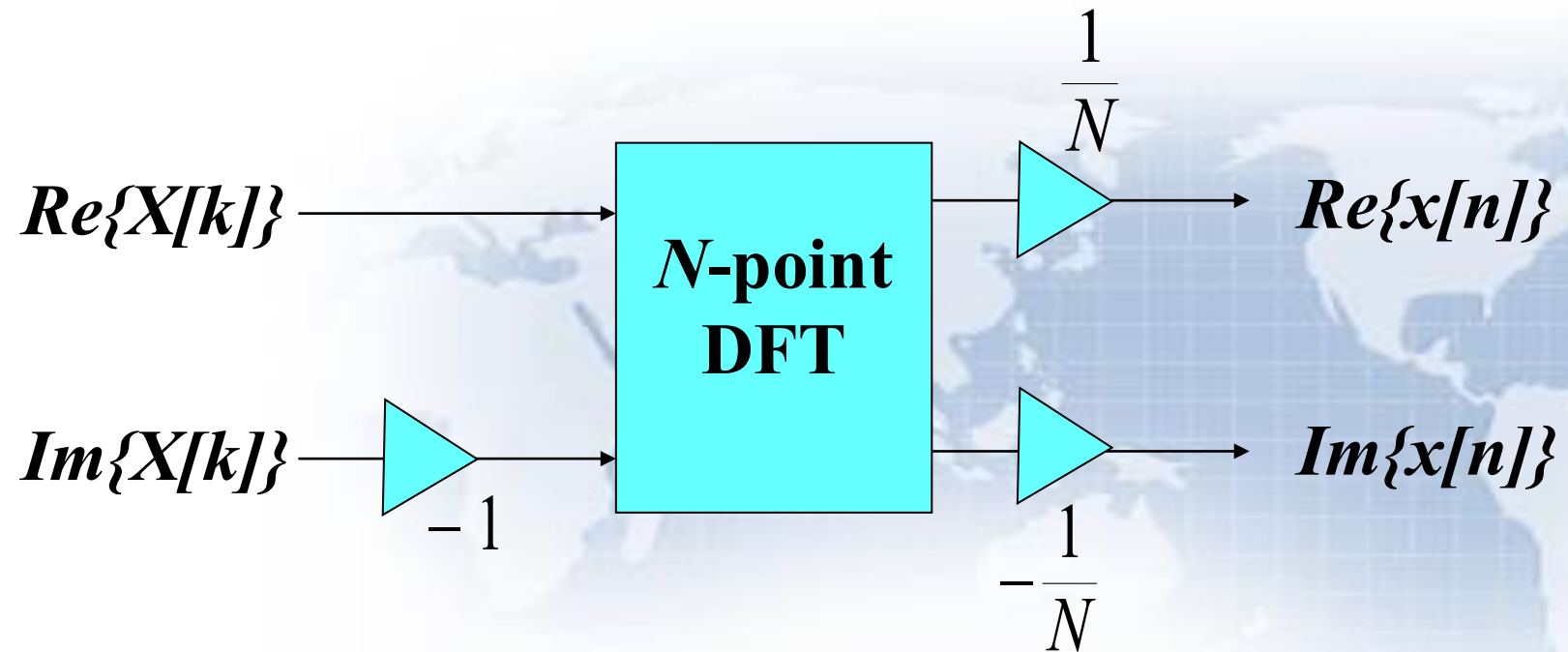
- **Right-hand side of above is the N -point DFT of $X^*[k]$.**
- **Desired IDFT $x[n]$ is then obtained as:**

$$x[n] = \frac{1}{N} \left\{ \sum_{k=0}^{N-1} X^*[k] W_N^{nk} \right\}^*$$

Inverse DFT Computation



Inverse DFT computation is shown below:





11.5 DFT and IDFT Computation Using MATLAB

- **Program 11_10.m**
 - **Spectral Analysis**
- **Program 11_11.m**
 - **Linear Convolution**

11.8 Number Representation